- 7. 自動お礼メール
  - 概要
  - 対応しているメール種別
  - 実装した主な機能
  - 実装コード（抜粋）
    - 送信本体（SMTP / Resend / audit 切替）
    - 寄付お礼テンプレート
    - 支援変更メールテンプレート
  - 検証・本番接続について
  - 画面・運用上で確認できるポイント
  - 実装上の意図
  - ご依頼内容との対応

7. 自動お礼メール

概要

会員さまの各種操作や Stripe からのイベントを起点に、 **お礼メール・お知らせメールを自動配信する仕組み**を実装いたしました。

すべての送信は、送信結果の成否にかかわらず `audit_logs` に 1 件ずつ履歴が残るため、 管理者画面から「いつ誰宛てにどの内容を送ったか」を追跡することができます。

対応しているメール種別

種別 (`NotifyKind`)	発火タイミング	主な内容
`donation_thanks`	単発寄付の Stripe 決済成功	ご寄付のお礼、金額、用途
`support_added`	支援申込の完了	対象馬・口数・月額
`support_changed`	口数 / 馬の変更	変更前後の比較、日割り精算の案内

種別 (NotifyKind)	発火タイミング	主な内容
support_canceled	支援停止（即時 / 期末）	終了予定日、これまでのお礼
payment_failed	invoice.payment_failed Webhook	決済失敗、マイページへの導線
plan_changed	会員種別変更	新プラン名、月額
booking_confirmed	見学会予約完了	イベント名、日時、集合場所
booking_canceled	見学会キャンセル	キャンセル受付の確認

実装した主な機能

- 送信経路は 3 種類から環境変数で切替可能
  - smtp（nodemailer 経由、Gmail SMTP 等）: 本番想定の既定値
  - resend（Resend HTTP API）: 大量配信時の切替用
  - audit: 配信せず audit_logs にのみ記録（ローカル検証用）
- 送信成否・件名・本文プレビュー・送信経路を audit_logs に保存
- ログ記録が失敗しても呼び出し元は失敗しない（安全側 catch）
- 署名ブロック（お問い合わせ先・マイページ URL）を全テンプレートに共通挿入
- Stripe Webhook や支援操作 API と接続し、イベント起点で自動送信できる構成を整備

実装コード（抜粋）

送信本体（SMTP / Resend / audit 切替）

```
// src/lib/notify.ts
export async function notify(payload: NotifyPayload): Promise<{
  sent: boolean; transport: string; error?: string;
}> {
  const transport = (process.env.NOTIFY_TRANSPORT ?? "smtp").toLowerCase();
  let result: { ok: boolean; error?: string } = { ok: false, error: "no transport"
};

  if (payload.to) {
```

```
    if (transport === "smtp") result = await sendViaSmtp(payload);
    else if (transport === "resend") result = await sendViaResend(payload);
    else result = { ok: false, error: "audit-only mode" };
  } else {
    result = { ok: false, error: "no recipient" };
  }

  // 送信結果を必ず audit_logs に残す
  try {
    const admin = createSupabaseAdminClient();
    await admin.from("audit_logs").insert({
      action: `notify.${payload.kind}`,
      target_table: "notifications",
      meta: {
        to: payload.to,
        to_name: payload.to_name ?? null,
        subject: payload.subject,
        preview: payload.body_text.slice(0, 400),
        transport,
        sent: result.ok,
        error: result.error ?? null,
      },
    });
  } catch {
    // logging must not break the caller
  }
  return { sent: result.ok, transport, error: result.error };
}
```

寄付お礼テンプレート

```
// src/lib/notify.ts
export function donationThanksTemplate(params: {
  name: string | null;
  amount: number;
}) {
  const who = params.name?.trim() || "ご支援者";
  return {
    subject: "【Retouch Members】ご寄付ありがとうございます",
    body_text:
      `${who}様\n\n` +
      `このたびは引退競走馬への温かいご寄付（${yen(params.amount)}）を賜り、誠にありがとうございます。\n` +
      `いただいたご支援は、馬たちのケア・見学会の運営にありがたく活用させていただきます。\n\n` +
      `本メールは寄付受付の確認としてお送りしております。` +
      signature(),
  };
}
```

支援変更メールテンプレート

```
export function supportChangedTemplate(params: {
  name: string | null;
  horseName: string;
  prevUnits: number; prevMonthly: number;
  newUnits: number; newMonthly: number;
}) {
  const who = params.name?.trim() || "ご支援者";
  const fmt = (n: number) => (Number.isInteger(n) ? `${n}口` : `${n.toFixed(1)}口
`);
  return {
    subject: `【Retouch Members】支援内容変更のお知らせ — ${params.horseName}`,
    body_text:
      `${who}様\n\n` +
      `${params.horseName}の支援内容を以下のとおり変更いたしました。\n\n` +
      `【変更前】 ${fmt(params.prevUnits)} / 月額 ${yen(params.prevMonthly)}\n` +
      `【変更後】 ${fmt(params.newUnits)} / 月額 ${yen(params.newMonthly)}\n\n` +
      `Stripeの仕様により、月の途中での変更は日割り計算にて差額が次回請求に反映されます。`
+
      signature(),
  };
}
```

検証・本番接続について

当方側では、検証用の送信設定をもとにメール機能の実装準備を進めております。 アプリケーション側では、寄付完了・決済失敗・支援変更・支援停止受付などのイベントから 自動で `notify()` が呼び出される構成を整えており、 送信先・件名・本文・送信成否を記録する仕組みまで実装済みです。

一方で、御社で実際に運用される送信元メールアドレスに切り替えるには、 SMTP 情報またはメールサービスのログイン情報が必要となります。 そのため、本番送信については、御社にて利用予定のメールアカウント情報をご共有いただき次第、 最終設定と実送信確認を行う流れとしております。

なお、お預かりしたメール関連情報は、 プロジェクト完了時にすべて御社へお戻しする前提で運用いたしますので、 その点はご安心いただければと存じます。

画面・運用上で確認できるポイント

本機能は、会員さまから直接「メール画面」が見えるタイプの機能ではなく、 各操作完了時に裏側で自動送信される仕組みです。 そのため、会員さま側では寄付完了画面や支援完了画面で 「確認メールをお送りしています」という案内文を表示し、 管理・運用側では `audit_logs` に記録された送信履歴をもとに、 いつ・誰に・どの内容を送ったかを追跡できる構成としております。

また、送信処理そのものが失敗した場合でも、 アプリケーション全体の主要処理が巻き戻らないようにしており、 たとえば寄付成功や支援変更の確定処理が、 メール送信エラーによって失敗扱いにならないよう、安全側で設計しております。

実装上の意図

自動お礼メールは、単に通知を送るためだけでなく、 会員さま・寄付者さまに「きちんと受け付けられた」という安心感をお届けする役割を担っています。 そのため、件名・本文ともにできるだけ平易な日本語とし、 操作完了後に不安を残さない文面に統一しています。

また、将来的に送信経路を Gmail SMTP から Resend へ切り替える可能性や、 本番環境・検証環境を分けて管理する必要も見据え、 送信処理は `notify()` に集約し、環境変数で切替可能な構成にしました。 これにより、Milestone 4 以降の本番切替・運用開始にも移行しやすい基盤となっております。

ご依頼内容との対応

いただいたご要望	対応状況
寄付・支援申込後に自動でお礼を送りたい	Webhook / サーバアクションから `notify()` を自動起動
決済失敗時にも会員へ通知したい	`invoice.payment_failed` で `paymentFailedTemplate` を送信
送信ログを後から追跡できるようにしたい	`audit_logs.action = 'notify.*'` に全件記録
本番では貴社のメールから送信したい	`NOTIFY_TRANSPORT` と SMTP 情報の差替で切替可能